Optimizing Kolmogorov-Arnold Networks: Accelerating Training in KANs

Abhishek Chaudhary*
Department of Computer Science
Columbia University
New York, USA
ac5003@columbia.edu

Neil Biju Kachappilly*
Department of Computer Science
Columbia University
New York, USA
nbk2122@columbia.edu

Greg Ou*
Department of Computer Science
Columbia University)
New York, USA
gyo2102@columbia.edu

***Abstract*—Kolmogorov-Arnold Networks (KANs) offer a compelling alternative to traditional Multi-Layer Perceptrons (MLPs) by replacing fixed nodal activations with learnable spline-based activation functions defined on edges. Rooted in the Kolmogorov-Arnold representation theorem, KANs deliver improved interpretability and superior performance in tasks such as scientific computing and symbolic regression, all while using fewer parameters and exhibiting favorable neural scaling behaviors. However, their deployment in practice is hindered by significant computational inefficiencies due to recursive B-spline evaluations, high-precision floating-point requirements, and hardware-unfriendly execution patterns. This paper addresses these challenges through a comprehensive optimization framework that integrates three key strategies: mixed-precision training using FP16/BF16 arithmetic, compilation acceleration via PyTorch TorchInductor and CUDA graph optimization, and architectural refinement through FastKAN, a recent KAN variant that substitutes splines with radial basis functions (RBFs). Our contributions include the first mixed-precision implementation of both KAN and FastKAN, quantized FastKAN variants, and performance-tuned compilation strategies leveraging autotuning and Triton CUDA graphs. These enhancements significantly reduce training and inference times, making KANs a more viable and efficient alternative to MLPs in modern deep learning applications.**

***Index Terms*—KAN, optimization, quantization, mixed precision, compilation**

I. Introduction

Kolmogorov-Arnold Networks (KANs) have emerged as a promising alternative to traditional Multi-Layer Perceptrons (MLPs), offering enhanced interpretability through their foundation in the Kolmogorov-Arnold representation theorem [1]. Unlike MLPs with fixed nodal activation functions, KANs employ learnable spline-based activation functions on network edges, enabling superior performance in scientific computing tasks and symbolic regression problems [2]. Recent analyses demonstrate that KANs achieve comparable accuracy to MLPs with significantly fewer parameters while maintaining faster neural scaling laws [3].

Despite these advantages, KANs face critical challenges in practical deployment due to computational inefficiencies. The recursive nature of B-spline calculations in traditional KAN implementations creates substantial bottlenecks in both training and inference phases [4]. Recent benchmarks reveal that even optimized KAN variants require 3-5× more computation time than equivalent MLP architectures for similar tasks [1]. This performance gap stems from three primary factors: (1) sequential evaluation of basis functions, (2) high-precision floating-point requirements, and (3) suboptimal hardware utilization patterns [3].

Our work addresses these challenges through a systematic optimization framework applying three approaches. First, we implement mixed-precision training techniques [5] using FP16/BF16 formats for activations and gradients while maintaining FP32 master weights. Second, we develop specialized compilation strategies using PyTorch's TorchInductor and CUDA graph optimizations. Third, we integrate recent advances in KAN acceleration through FastKAN's radial basis function approximations [2].

The key contributions of this paper include: KAN with mixed precision, FastKAN with mixed precision, quantized FastKAN, and all of the above compiled with max autotune or triton cuda graphs.

II. Literature Review

A. Review of Relevant Literature

KANs were introduced in the seminal paper titled "KAN: Kolmogorov-Arnold Networks" [1], the fundamental literatuyre motivating our work in this paper. As mentioned in the introduction, these networks employ learnable spline-based activation functions defined on edges, contrasting sharply with the fixed activation functions traditionally placed at nodes in MLP architectures. This innovative approach positions KANs as highly flexible, interpretable, and capable of capturing intricate nonlinear relationships in data.

At the heart of KANs is their reliance on spline-based representations. Unlike standard neural networks, where fixed activation functions (such as ReLU, sigmoid, or tanh) constrain expressivity, KANs adopt B-spline activations that dynamically adjust during the training process [1]. These spline functions, defined piecewise with polynomial segments and interconnected at nodes called knots, allow KANs to approximate complex functional mappings accurately and smoothly. The implementation provided by the GitHub repository "KindXiaoming/pykan" [1] offers a robust reference framework for researchers and practitioners aiming to experiment with and

*All authors contributed equally to this work.

further develop spline-based representations within neural network architectures. This implementation has significantly facilitated the empirical validation and further exploration of KANs' theoretical propositions. We use the same GitHub repository as our starting point to conduct the optimizations detailed later in the paper.

KANs' theoretical underpinning, the Kolmogorov-Arnold theorem, asserts that any multivariate continuous function can be represented through compositions and superpositions of continuous univariate functions. This powerful result motivates the spline-based construction, as splines naturally fulfill these theoretical conditions, thereby granting KANs intrinsic flexibility and interpretability [1]. Empirical evaluations have shown that KANs can achieve improved accuracy over traditional MLPs on various benchmark tasks, particularly due to their ability to adaptively capture and represent data complexities.

Building upon the foundational concepts of KANs, the subsequent work "Kolmogorov-Arnold Networks are Radial Basis Function Networks" introduces an optimized variant termed FastKAN [2]. FastKAN replaces the spline-based B-spline functions utilized in traditional KAN architectures with Gaussian Radial Basis Functions (RBFs). This offers significant potential for further optimizations that were previously hindered by the complexity of optimizing spline function parameters. While this substitution significantly enhances computational efficiency—addressing one of the critical challenges associated with spline computations—it introduces new considerations. Notably, FastKAN's reliance on RBFs simplifies the optimization landscape, yet initial studies indicate limited exploration of more complex optimization scenarios [2]. Thus, while promising in terms of speed and efficiency, FastKAN may sacrifice some of the expressive power and interpretability inherent in spline-based KANs.

Despite the computational advantages introduced by FastKAN, the core strength of original KANs—interpretability through spline parameters and adaptability in capturing nonlinearities—remains a compelling rationale for their continued development and optimization [1]. Future research must carefully balance efficiency gains offered by variants like FastKAN with the expressive advantages of spline-based approaches. Moreover, exploring hybrid or adaptive architectures that selectively incorporate spline and RBF-based representations could potentially address current limitations, enabling broader applicability and optimized performance across diverse tasks.

Overall, KANs represent a significant innovation in neural network design, marrying theoretical rigor from classical mathematics with modern computational approaches. Their spline-based activations, grounded in the Kolmogorov-Arnold theorem, offer substantial benefits over traditional MLPs regarding flexibility, interpretability, and accuracy [1]. While variants such as FastKAN illustrate pathways to greater computational efficiency [2], they also underscore the importance of thorough empirical investigation into optimization landscapes. Continued research in optimizing KANs, exploring hybrid approaches, and expanding empirical validation will be critical to fully realizing their potential impact across machine learning and artificial intelligence domains.

METHODOLOGY

B. Datasets

We employed two synthetic datasets based on mathematical formulations to evaluate the representational efficiency and computational performance of KANs compared to standard MLPs.

The first dataset is a two-dimensional regression problem defined by the equation:

$$f(x, y) = \exp\left(\sin(\pi x) + y^2\right) \tag{1}$$

This function combines periodic behavior with exponential growth, providing a nontrivial test of a model's ability to represent nonlinear relationships. The dataset was used to directly compare training and validation losses between KANs and parameter-matched MLPs, isolating differences in expressive power.

The second dataset is derived from the Black-Scholes call option pricing model, a widely-used equation in quantitative finance. It predicts the theoretical price of a European call option as a function of five input parameters:

- Time to maturity T,
- Stock price S,
- Strike price K,
- Risk-free interest rate r, and
- Volatility σ.

The formula for the Black-Scholes price of a European call option is given by:

$$C(S, K, T, r, \sigma) = S\Phi(d_1) - Ke^{-rT}\Phi(d_2) \tag{2}$$

where:

$$d_1 = \frac{1}{\sigma\sqrt{T}}\left[\ln\left(\frac{S}{K}\right) + \left(r + \frac{\sigma^2}{2}\right)T\right] \tag{3}$$

$$d_2 = d_1 - \sigma\sqrt{T} \tag{4}$$

and $\Phi(\cdot)$ denotes the cumulative distribution function (CDF) of the standard normal distribution.

We generated synthetic data by evaluating this formula across a realistic range of parameter values. This dataset was primarily used for benchmarking training and inference times between MLP and KAN models, serving as a baseline for measuring the speedup achieved through our KAN optimization techniques.

By utilizing mathematically defined functions, both datasets offer controlled complexity and enable clear, interpretable comparisons between model architectures.

C. Optimization Procedures

In this paper, we attempt to optimize both training and inference times using quantization, mixed precision, and compilation techniques.

1) Mixed Precision Training: We applied mixed-precision training using PyTorch's `torch.cuda.amp` utilities to reduce memory usage and increase computational throughput. Forward passes and loss computations were wrapped with `autocast()`, enabling automatic casting of eligible operations to FP16 or BF16 precision. Gradients were scaled using `GradScaler` to prevent underflow during backpropagation, and updates were performed using the Adam optimizer with master weights retained in FP32.

2) Compilation with TorchInductor and Triton: To accelerate execution, models were compiled using `torch.compile()` with TorchInductor as the backend. TorchInductor applies graph-level optimizations such as operation fusion, memory planning, and backend-specific kernel generation. Further improvements were achieved by enabling Triton-based CUDA graph compilation, which reduces kernel launch latency and CPU-GPU synchronization overhead by statically capturing the execution graph. These compilation strategies provided significant runtime improvements, particularly for FastKAN.

3) Quantization: We explored both post-training quantization (PTQ) and quantization-aware training (QAT) to reduce inference latency and model size. For the original KAN, PTQ was implemented using both PyTorch's `QuantStub` and a manual scaling approach for weights and activations. However, due to the nonlinear nature of spline operations, PTQ proved unstable in certain configurations.

For FastKAN, we applied QAT using the `x86.qconfig` scheme, enabling training-time simulation of quantized behavior. This allowed the model to learn robustness to quantization effects. The resulting quantized model was compiled with TorchInductor's `max-autotune` mode enabled, activating operator-level autotuning, aggressive kernel fusion, and advanced graph rewrites to further enhance performance.

D. Profiling tools and methods

We used PyTorch's built-in profiler to analyze runtime performance across all KAN variants. Initial profiling of the vanilla KAN revealed that the model was compute-bound, with the `coef2curve` operation dominating execution time. This informed our optimization focus, particularly in FastKAN. We then compared all models qualitatively and quantitatively, using profiling data to measure speedups reported in our experiments. At a larger input size of 500,000, the workload became I/O-bound, with the trace dominated by memory copy operations, indicating that data transfer, rather than computation, became the primary bottleneck at scale.

E. Evaluation Metrics

To assess both the effectiveness and trade-offs of our optimization techniques, we primarily measured changes in training/validation loss and execution time. Loss was used to detect any degradation in model performance due to quantization, mixed precision, or compilation, while execution time—recorded during both training and inference—served as the main metric for evaluating the effectiveness of our optimizations. Together, these metrics provided a balanced view of optimization quality, ensuring that speedups were not achieved at the cost of significant performance loss.

Experimental Results

F. Experimental Setup

To evaluate the representational efficiency and runtime performance of Kolmogorov-Arnold Networks (KANs), we conducted a series of controlled experiments. The first compared them to traditional Multi-Layer Perceptrons (MLPs) with equivalent parameter counts. We constructed an iterative MLP that adjusted its layer widths and depth to closely match the number of trainable parameters. Both models were trained on the same dataset using identical optimization settings, and we recorded training and validation losses to assess their expressiveness and generalization. This allowed us to empirically verify that KANs can achieve lower loss with fewer or equivalent parameters, consistent with claims of improved representational capacity.

In addition to accuracy comparisons, we benchmarked the training and inference times of both architectures to highlight the computational overhead introduced by KAN's spline-based structure. These baseline timings were used as a reference point for evaluating the effectiveness of subsequent optimizations.

To address the performance bottlenecks observed in unoptimized KANs, we introduced a multi-stage optimization pipeline. First, we applied mixed-precision training using FP16 or BF16 formats for activations and gradients, while preserving FP32 master weights to maintain numerical stability. This aimed to reduce memory usage and improved throughput without compromising accuracy. Next, we leveraged PyTorch's TorchInductor backend to compile KAN models into optimized computation graphs. We enabled full graph compilation under static shape assumptions, which allowed us to take advantage of aggressive kernel fusion and graph-level memory planning. Triton-based CUDA graphs were also utilized to further reduce kernel launch overhead and improve low-level execution efficiency.

To mitigate the spline evaluation bottleneck more directly, we transitioned from traditional KANs to FastKAN, a variant that replaces B-spline activations with Gaussian radial basis functions (RBFs). FastKAN's architectural simplicity enabled improved GPU utilization and faster evaluation, especially when paired with the same mixed-precision and compilation strategies.

Finally, we applied quantization-aware training (QAT) to the FastKAN models, simulating integer inference during training to produce low-precision, deployable models. These quantized models were then compiled using TorchInductor's max-autotune mode, which includes operator-level autotuning, advanced graph rewriting, and aggressive kernel fusion. This final optimization stage yielded significant speedups with minimal accuracy degradation, demonstrating the feasibility of deploying KAN-style models in latency-sensitive environments.

Through this series of experiments, we systematically evaluated the trade-offs between accuracy, interpretability, and efficiency in KANs and their optimized variants, establishing a practical foundation for their broader application in high-performance deep learning tasks. An example trace is included in the appendix (see Fig. 8).

G. Performance Comparison

MLP had a $46.8\times$ the inference speed of the Vanilla KAN and established the benchmark speecup for input size of 10,000. The figures below compare models across different optimizations training/inference times and degradation in the model compared to the Vanilla KAN.

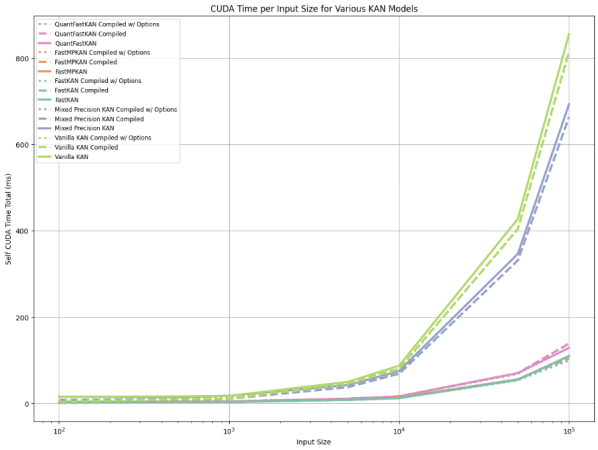

Fig. 1. CUDA time vs input size for different optimizations.

Figure 1 presents the CUDA times for varying input sizes to the model variations. All models of similar type are consistently colored.

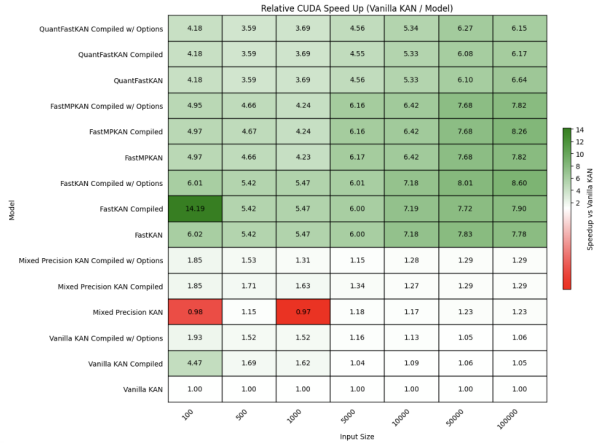

Fig. 2. Relative training time speedup for different optimizations.

Figure 2 presents a comprehensive comparison of relative CUDA speedup across various KAN model variants and optimization strategies, measured as the ratio of Vanilla KAN training time to that of each tested model for a range of input sizes.

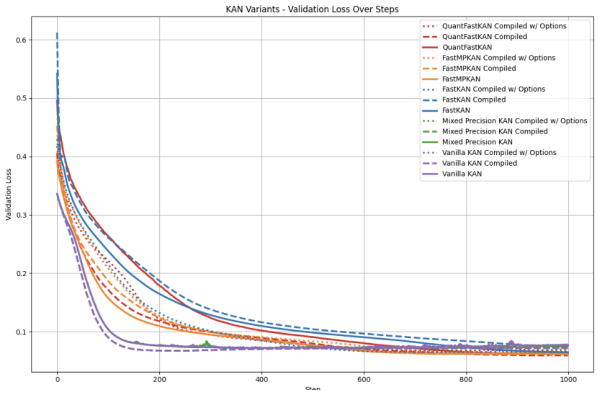

Fig. 3. Validation loss graph of different optimizations.

Figure 3 illustrates the validation loss trajectories for all evaluated KAN variants over the course of 1,000 training steps. Each curve represents a distinct model and optimization configuration.

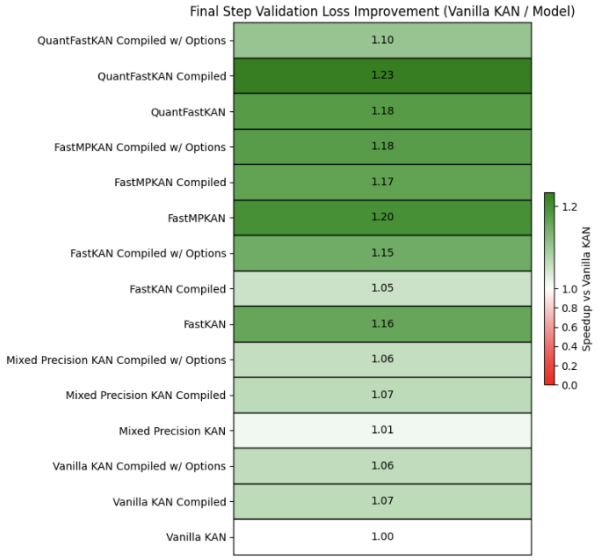

Fig. 4. Relative validation time speedup for different optimizations.

Figure 4 summarizes the improvement in final step validation loss for each KAN variant, expressed as the ratio of Vanilla KAN loss to that of the tested model. Bars shaded in deeper green indicate greater improvement relative to the Vanilla KAN baseline.

Figure 5 displays the relative inference speedup of each KAN variant compared to the Vanilla KAN baseline for an input size of 10,000. The speedup is calculated as the ratio of Vanilla KAN inference time to that of each model, with higher values indicating greater acceleration.

Discussion

H. Analysis/Interpretation of Results

Our experimental results reveal several important trends across optimization methods, model variants, and input sizes.

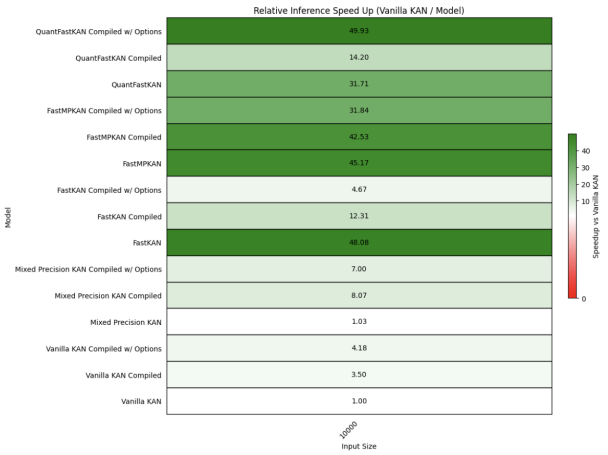

Fig. 5. Relative inference time speedup for different optimizations.

Compilation produced a measurable training speedup for both KAN and FastKAN models. While this benefit diminished with larger input sizes, it still offered a consistent advantage in reducing runtime during training. Mixed precision, in contrast, became increasingly effective as input size grew, offering greater throughput and lower memory overhead at scale.

Quantization introduced additional overhead during training, particularly for FastKAN, due to unsupported operations such as radial basis function computations not yet optimized for quantized execution. As expected, this led to slower training times. However, when quantized FastKAN was compiled with TorchInductor, it achieved a 1.04x speedup over its unquantized counterpart. This suggests that, despite current limitations, further gains are achievable through architectural refinement and more mature quantization support.

FastKAN compiled with all optimizations (mixed precision, full graph compilation, and Triton CUDA graphs) was consistently the fastest model variant, aside from an outlier observed at input size 100. At an input size of 100,000, mixed-precision FastKAN achieved a 1.078x speedup over the base FastKAN, compared to a smaller 1.005x gain at lower input sizes. These results reinforce the scalability of our optimization strategy.

Across all experiments, models converged in validation loss within 1000 iterations when trained on inputs of size 10,000 with the same learning rate. An input size of 10,000 was selected for validation and inference timing experiments, as it maintained near-maximum GPU utilization for the most computationally intensive model. Vanilla KAN converged more rapidly than its optimized variants in early training stages, likely due to the expressive power of its spline-based representation. Nevertheless, all models reached similar loss values by the end of training, and in some seeded runs, the optimized variants slightly outperformed the vanilla model. Compilation was observed to slightly improve convergence behavior for both vanilla KAN and Quantized FastKAN.

Inference time improvements were particularly significant. Compiled vanilla KAN models achieved at least a 3.5x speedup, demonstrating the substantial efficiency gains made possible through kernel fusion and graph optimization. While mixed precision led to modest improvements for FastKAN, its performance was limited by `autocast()` overhead and increased kernel launches, suggesting that larger model sizes are necessary to fully realize its benefits.

Although quantized FastKAN incurred overhead during training, compilation enabled it to surpass regular FastKAN in inference speed, achieving a 1.04x improvement. This indicates strong potential for quantization, especially with further architectural tailoring and improved compiler support for RBF operations.

Overall, as input sizes increased, KAN-based models exhibited more pronounced performance gains. The combination of mixed precision, compilation, and quantization achieved improved training and inference efficiency without sacrificing convergence or accuracy. These findings confirm that high-performance machine learning (HPML) techniques can be successfully applied to nonstandard architectures like KANs and FastKAN, with further gains possible through quantization-aware design and hardware-aligned compilation strategies.

I. Challenges and Limitations

While this work demonstrates meaningful progress in optimizing Kolmogorov-Arnold Networks (KANs), several challenges and limitations remain due to the novelty and structural uniqueness of these architectures.

KANs, despite being proposed as an alternative to traditional Multi-Layer Perceptrons (MLPs), are still under theoretical scrutiny. Recent discussions in the literature suggest that KANs may, under certain conditions, be reducible to reparameterizations of MLPs, which raises questions about their distinctiveness in terms of representational capacity. This ambiguity presents a conceptual challenge in both evaluation and benchmarking, especially when attempting to attribute observed improvements solely to architectural innovation.

Additionally, KANs rely on spline and radial basis function (RBF) representations, which are not inherently compatible with many high-performance machine learning (HPML) optimization techniques. Unlike standard linear layers with fixed activation functions, these learnable functions introduce complexity in both numerical behavior and software integration. Existing PyTorch abstractions and compilation backends are often tuned for linear algebra operations and do not natively support the kinds of custom differentiable function evaluations used in KANs. As such, applying common wrappers like quantization-aware training (QAT), mixed precision, or TorchInductor requires bespoke handling and frequent workarounds.

Quantization, in particular, poses a unique challenge. While quantizing linear weights in MLPs is well-established, quantizing spline or RBF-based transformations is nontrivial. These components often involve floating-point arithmetic that is sensitive to resolution, and a naive quantization can degrade model performance or break differentiability. Moreover, combining quantization with mixed precision in FastKAN requires careful management of type casting between `float32`

and `float16`, especially within custom modules like `SplineLinear` and `RadialBasisFunction`.

Toolchain limitations further compound the difficulty. For instance, TorchInductor's configuration using `dynamic=False`, `fullgraph=True`, and `triton.cudagraphs=True`—which provides optimal graph-level compilation—is not compatible with PyTorch's QAT workflows. This restriction forces trade-offs between model portability, compilation depth, and optimization aggressiveness.

Finally, there is minimal precedent in the literature or tooling ecosystem for performing HPML-style optimizations on architectures that deviate so fundamentally from standard MLPs. This novelty both motivates and limits our work, requiring significant engineering effort and experimentation to adapt existing infrastructure to support spline- and RBF-based networks.

These challenges emphasize the need for continued toolchain advancement and architectural co-design to support emerging nonstandard neural models like KANs.

J. Future Directions

While our current work demonstrates the viability of optimizing Kolmogorov-Arnold Networks (KANs) through mixed precision, compilation, and quantization, several promising avenues remain for future exploration. One such direction involves improving the quantization pipeline—particularly for inference—by implementing custom quantized operators for KAN-specific components such as spline or radial basis function evaluations. This could further reduce latency and bring quantized KANs closer in speed to highly optimized MLP baselines.

In addition, optimizing the MLP baseline itself could establish a stronger benchmark for assessing the performance ceiling of KAN variants. Techniques such as kernel fusion, layer folding, and advanced autotuning could be applied to MLPs to better contextualize the performance gap and guide future improvements in KAN architecture and compilation strategies.

Further research into the compilation and quantization of KAN graphs may also yield additional speedups. Enhanced graph rewriting rules, support for custom fused kernels, and static shape analysis tailored to KAN structures could unlock more efficient execution paths on modern accelerators.

Finally, pruning strategies for KANs represent an underexplored yet promising direction. Benchmarking inference time and accuracy after structured or unstructured pruning of spline- or RBF-based KAN models could reveal new trade-offs between model sparsity, interpretability, and execution efficiency. Collectively, these directions point toward a robust roadmap for advancing the practical deployment of KANs in high-performance machine learning systems.

Conclusion

Summary of Findings

This work investigates the viability of Kolmogorov-Arnold Networks (KANs) as efficient alternatives to Multi-Layer Perceptrons (MLPs) by targeting one of their primary limitations: computational inefficiency. We demonstrated that, when matched for parameter count, our optimized KANs consistently achieve superior predictive performance relative to MLPs. We find that while vanilla KANs suffer from significant overhead during both training and inference due to the complexity of their spline-based activation functions, alternative activation functions (like in FastKANs) can be applied along with traditional optimization techniques to increase GPU utilization and dramatically reduce latency.

Through systematic application of high-performance optimization techniques—including mixed precision training, TorchInductor graph compilation, Triton CUDA graph integration, and quantization-aware training—we achieved substantial reductions in runtime. In our most optimized configuration, we observed a peak inference speedup of 48.08x over the unoptimized baseline KAN, exceeding the performance of a comparable MLP baseline by 1.03x.

While the effectiveness of these techniques varies depending on model size and input dimensionality, our experiments confirm that KAN-specific optimizations, particularly when applied to FastKAN, can yield meaningful performance improvements without compromising model accuracy.

Contributions

This paper makes the following contributions:

- We implement and evaluate multiple high-performance training and inference strategies tailored for KANs, including mixed precision training, TorchInductor compilation, and quantization-aware training. These optimizations were applied to both vanilla KAN and FastKAN models.
- We demonstrate that these optimizations substantially reduce computational overhead, observing significant speedup over the baseline KAN and bring optimized KAN performance at least on par with a comparable MLP in inference time.
- We develop and release a lightweight open-source codebase that supports reproducible experimentation with KAN optimization, including our custom dataset generator based on the Black-Scholes option pricing model.[1]

These results contribute to closing the gap between theoretical expressiveness and practical usability of KANs, and lay the groundwork for future research into deployable KAN-based architectures.

Acknowledgment

Thank you to HPML Professor Kaoutar El Maghraoui and the TAs for their support throughout the execution of this research project.

[1] https://github.com/neilkach/optimized-kan

References

[1] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljacic, T. Y. Hou, and M. Tegmark, "Kan: Kolmogorov-arnold networks," *arXiv preprint arXiv:2404.19756*, 2024.

[2] Z. Li, "Kolmogorov-arnold networks are radial basis function networks," *arXiv preprint arXiv:2405.06721*, 2024.

[3] L. Chen, "Matrixkan: Parallelized kolmogorov-arnold network," *arXiv preprint arXiv:2502.07176*, 2025.

[4] A. Moradzadeh, "Ukan: Unbound kolmogorov-arnold network accompanied with accelerated library," *arXiv preprint arXiv:2408.11200*, 2024.

[5] P. Micikevicius, S. Narang, J. Alben *et al.*, "Mixed precision training," *arXiv preprint arXiv:1710.03740*, 2018.

Appendix A
MLP vs Vanilla KAN

In figures 6 and 7, we examine MLPs vs vanilla KAN to understand the scope of our problem. The below three images exhibit the results of said experiment.

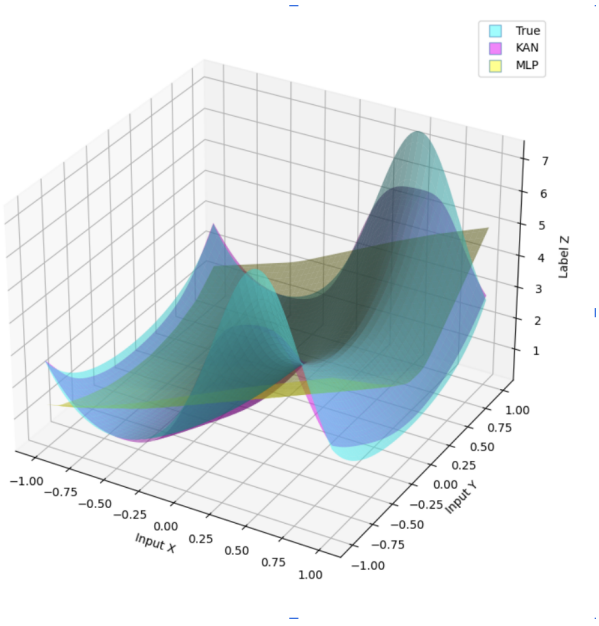

Fig. 6. Comparing learned $f(x, y) = \exp\left(\sin(\pi x) + y^2\right)$

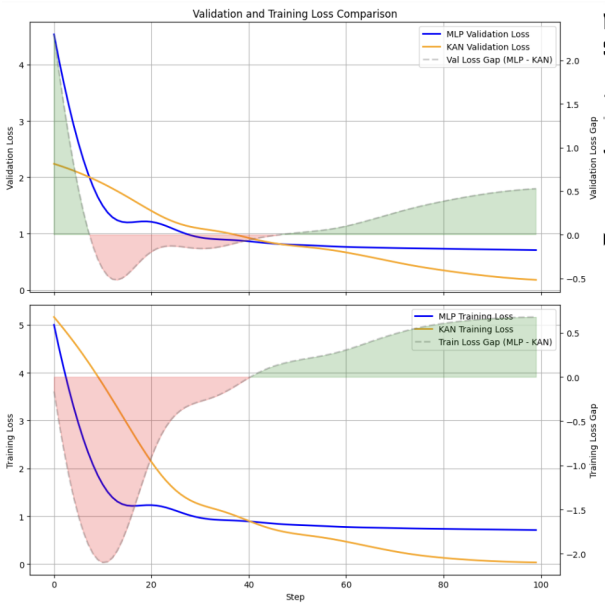

Fig. 7. Training and validation loss comparison between KAN and MLP on the two-variable exponential dataset.

Finally, we examine vanilla KAN profiling to observe its most computationally intense aspect.

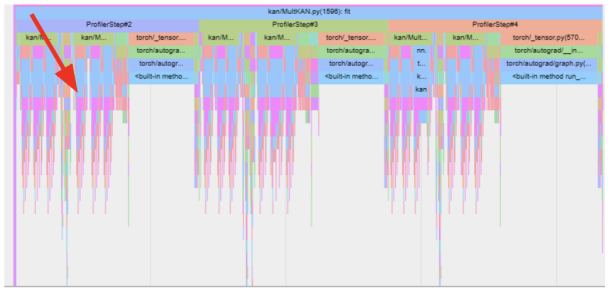

Fig. 8. Profiling different aspects of vanilla KAN on the two-variable exponential dataset.